

# Finlora Finance Flow — Fraud Detection and Risk Scoring

---

## Final Documentation and Handover Summary

Project type: Machine-learning fraud-risk scoring prototype

Primary model: Random Forest

Baseline model: Logistic Regression

Deployment: Streamlit

Operating threshold: 0.50 fraud probability

Dataset: 126,000 transactions across approximately 7,200 accounts

## 1. Executive Summary

Finlora Finance Flow is a fintech transaction-monitoring use case in which fixed-threshold rules can generate unnecessary alerts and may fail to identify more subtle fraudulent behaviour as transaction volume and behavioural complexity increase.

This project developed a machine-learning fraud-risk scoring prototype that assigns each transaction a probability of fraud rather than relying solely on a binary rule trigger. The resulting score can be used to prioritise transactions for fraud-analyst review.

The solution covers:

1. Data cleaning and validation.
2. Exploratory data analysis.
3. Behavioural and temporal feature engineering.
4. Stratified model development.
5. Class-imbalance handling using SMOTE on training data only.
6. Logistic Regression as an interpretable baseline.
7. Random Forest as the primary prototype model.
8. Model evaluation using accuracy, precision, recall, F1-score and ROC-AUC.
9. Fraud-probability/risk scoring.
10. Streamlit deployment for interactive transaction assessment.

The prototype is designed to support analyst review, not automatically block transactions.

## 2. Business Problem

Finlora processes transactions across multiple account types, channels, currencies and customer behaviours. A purely rule-based monitoring approach can become difficult to manage when transaction volume increases.

The project therefore addresses the question:

> Can transaction and account-level behavioural information be used to estimate the probability that a transaction is fraudulent and help analysts prioritise high-risk transactions?

The proposed solution augments rule-based monitoring with a continuous machine-learning risk score.

## 3. Project Objectives

### Primary objective

Develop an explainable fraud-risk scoring model capable of estimating the probability of fraud for individual transactions.

### Supporting objectives

- Clean and validate transaction and account data.
- Identify important behavioural patterns associated with fraud.
- Engineer transaction-level risk indicators.
- Handle the strong class imbalance in fraud labels.
- Establish an interpretable Logistic Regression baseline.
- Develop a non-linear Random Forest model.
- Compare the models using fraud-relevant evaluation metrics.
- Produce transaction-level risk scores.
- Provide an interactive Streamlit interface.
- Support analyst prioritisation rather than automatic transaction blocking.

## 4. Dataset

The modelling dataset contains:

- 126,000 transactions
- Approximately 7,200 accounts
- 3,352 fraudulent transactions
- 122,648 legitimate transactions
- Fraud prevalence: approximately 2.66%
- Legitimate transactions: approximately 97.34%

This represents a strongly imbalanced binary-classification problem.

Because of the imbalance, accuracy alone is not sufficient for judging model usefulness. Precision, recall, F1-score and ROC-AUC are therefore considered alongside accuracy.

---

## 5. Data Preparation

The data-cleaning workflow followed:

Load → Standardise → Profile → Clean → Validate → Save

Key preparation activities included:

- Standardising column names.
- Inspecting data types and missing values.
- Checking duplicate records.
- Converting timestamp fields to datetime.
- Cleaning text whitespace.
- Standardising categorical values.
- Handling missing merchant information.
- Preserving missing device information as an explicit unknown state where appropriate.
- Validating fraud labels.
- Investigating unusual negative 30-day average values.
- Checking duplicate account-level fields created during the transaction/account merge.
- Removing redundant `\_x` and `\_y` columns after validation.
- Saving cleaned datasets for subsequent analysis.

### Important data-quality decision

The dataset contained negative values in `avg\_transaction\_amount\_30d`. These were not simply replaced with zero, absolute values or missing values. Instead, they were flagged because changing them without establishing their source could introduce bias or hide a data-generation issue.

A `negative\_avg\_flag` was retained as an indicator.

## 6. Exploratory Data Analysis

EDA examined fraud behaviour across:

- Transaction amount.
- Amount-to-average ratio.
- Transaction velocity.
- Device status.
- KYC tier.
- Account type.
- Transaction channel.
- Merchant category.
- Transaction status.
- Cross-border activity.
- Transaction country.

- Hour of day.
- Day of week.
- Account age.
- Currency.
- Numerical relationships with fraud.

### Important observed risk signals

The analysis showed that fraudulent transactions tended to have higher transaction amounts and higher amount-to-account-average ratios than legitimate transactions.

Transaction velocity was also a useful behavioural signal. The independently calculated velocity feature matched the existing feature with 100% agreement.

New-device activity showed a higher observed fraud rate than existing-device activity, while unknown device status was retained separately rather than being treated as an existing device.

Merchant category, account type, transaction channel, KYC tier and cross-border activity also showed differences in observed fraud rates.

These findings are treated as risk associations rather than causal relationships.

## 7. Feature Engineering

The feature-engineering stage created variables designed to represent transaction size, account behaviour, velocity, device context and temporal behaviour.

### Main engineered features

- ``amount_to_avg_ratio``
- ``transaction_velocity_1h``
- ``log_amount``
- ``log_amount_to_avg_ratio``
- ``hour_sin``
- ``hour_cos``
- ``day_of_week_num``
- ``day_sin``
- ``day_cos``
- ``high_amount_ratio_flag``
- ``high_velocity_flag``
- ``new_device_flag``
- ``cross_border_flag``
- ``account_age_group``
- ``device_status_unknown``
- ``negative_avg_flag``

### Key behavioural features

Amount-to-average ratio

Measures how unusual the transaction amount is relative to the account's historical transaction behaviour.

Transaction velocity

Captures transaction activity within a recent one-hour window.

New-device indicator

Identifies activity from a device classified as new.

Cross-border indicator

Identifies transactions where transaction country differs from the customer's home country.

Temporal cyclical features

Sine/cosine transformations allow models to represent the cyclical nature of hours and days.

## 8. Features Excluded from Modelling

The following variables were excluded:

| Feature                | Reason                                                            |
|------------------------|-------------------------------------------------------------------|
| `transaction_id`       | Identifier, not behavioural information                           |
| `account_id`           | Identifier and potential memorisation risk                        |
| `account_holder_name`  | Personally identifiable information and unnecessary for modelling |
| `device_id`            | High-cardinality identifier                                       |
| `description`          | NLP outside the current LR/RF scope                               |
| `timestamp`            | Replaced by engineered temporal variables                         |
| `account_created_date` | Account age information already represented                       |
| `status`               | Potential target leakage                                          |
| `is_fraud`             | Target variable                                                   |

The exclusion of `status` is particularly important because a fraud model should ideally use information available when the transaction is assessed, rather than information that may be produced after processing or investigation.

## 9. Model Development

### Train/test split

A stratified 75/25 train-test split was used.

- Training set: 94,500 transactions.
- Test set: 31,500 transactions.

Stratification preserved the fraud/legitimate class proportions across the split.

## Categorical encoding

Categorical variables were one-hot encoded using an encoder configured to ignore previously unseen categories during transformation.

## Class imbalance

SMOTE was applied only to the training data.

Before SMOTE:

- Legitimate: 91,986
- Fraud: 2,514

After SMOTE:

- Legitimate: 91,986
- Fraud: 91,986

The test set was deliberately left untouched so that final evaluation represented the original class distribution.

## 10. Models

### Logistic Regression

Logistic Regression was used as the interpretable baseline.

It provides a useful reference point and produces probability estimates that are comparatively straightforward to interpret.

Scaling was applied for Logistic Regression.

### Random Forest

Random Forest was developed as the primary non-linear prototype.

Configuration:

- ``n_estimators = 300``
- ``max_depth = 10``
- ``random_state = 42``
- ``n_jobs = -1``

Random Forest does not require feature scaling, so it was trained on the encoded SMOTE-balanced data.

## 11. Model Performance

The final evaluation produced the following results:

| Metric    | Logistic Regression | Random Forest |
|-----------|---------------------|---------------|
| Accuracy  | 0.9452              | 0.9768        |
| Precision | 0.3055              | 0.5492        |
| Recall    | 0.8329              | 0.7196        |
| F1-score  | 0.4470              | 0.6229        |
| ROC-AUC   | 0.9389              | 0.9356        |

## Model selection

Random Forest was selected as the primary prototype because it provides stronger overall classification performance for the intended operational fraud-review use case, particularly in:

- Accuracy.
- Precision.
- F1-score.

Logistic Regression remains valuable as the interpretable benchmark and achieved higher recall and slightly higher ROC-AUC.

Therefore, Random Forest should not be described as superior on every metric. The selection is based on the overall operational balance of performance.

## 12. Risk Scoring

The Random Forest model produces a fraud probability between 0 and 1.

This is converted to a percentage:

Risk Score = Fraud Probability  $\times$  100

For example:

- Probability = 0.08  $\rightarrow$  Risk score = 8%
- Probability = 0.50  $\rightarrow$  Risk score = 50%
- Probability = 0.92  $\rightarrow$  Risk score = 92%

## Deployment operating threshold

The current deployment uses the standard:

0.50 probability threshold

Transactions with:

fraud probability  $\geq$  0.50

are prioritised for analyst review.

The project does not use the earlier 70%, 80% or 90% target-recall thresholds as the deployment rule.

### 13. Risk Categories

The prototype uses:

| Fraud Probability | Risk Category | Action                               |
|-------------------|---------------|--------------------------------------|
| < 0.30            | Low Risk      | No immediate review                  |
| 0.30–<0.50        | Medium Risk   | Monitor / potential secondary review |
| ≥ 0.50            | High Risk     | Prioritise analyst review            |

The 0.50 threshold is the current operating threshold. The 0.30 and 0.50 category boundaries are prototype communication bands and should be recalibrated using operational validation data before production use.

### 14. Deployment Architecture

The deployment workflow is:

The saved deployment package contains:

- Trained Random Forest model.
- Fitted preprocessing pipeline.
- Model feature list.
- Operating threshold.

### 15. Streamlit Application

The Streamlit application provides an interactive interface titled:

Finlora Fraud Detection and Risk Scoring

It accepts transaction information such as:

- Transaction amount.
- 30-day average transaction amount.
- Transaction velocity.
- Account age.
- Personal spending baseline.
- Hour/day.
- Account type.
- KYC tier.
- Merchant category.
- Channel.
- Currency.
- Transaction country.
- Home country.
- Cross-border status.
- New-device status.
- Other risk indicators.



The application calculates engineered features before passing the transaction through the saved preprocessor and Random Forest model.

It then displays the estimated fraud probability and provides a review decision based on the configured threshold.

The current Streamlit code loads the saved deployment package, performs feature engineering, transforms the transaction and obtains the Random Forest probability. [filecite\turn2file0\L7-L20](#)  
[filecite\turn2file0\L198-L220](#)

The application currently displays a high-risk analyst-review message when the probability meets the configured threshold and a low-risk message otherwise. [filecite\turn2file0\L351-L397](#)

## 16. Project File Structure

Recommended final repository structure:

## 17. Notebook-to-Python Handover

### Keep in Jupyter Notebook

The following remain primarily in the notebooks:

- Data cleaning.
- EDA.
- Feature engineering experiments.
- Model training.
- Model comparison.
- Confusion matrices.
- Feature importance.
- Research analysis.
- Model-development documentation.

### Move to Python

The following should be maintained as reusable Python code:

- Loading the deployment package.
- Transaction scoring.
- Review-queue generation.
- Analyst outcome logging.
- Monitoring/retraining utilities.
- Streamlit application.

This separates experimentation from deployment.

## 18. Deployment Dependencies

The deployment environment should contain compatible versions of the libraries used to create the model package.

Core dependencies include:

The exact package versions used for the final deployment environment should be frozen in `requirements.txt` after the working environment has been verified.

Example:

Do not invent versions in the final repository. Record the versions from the verified deployment environment.

## 19. Running the Streamlit Application

From the project root:

The application should load:

The deployment package path in the current application is configured to this location.

## 20. Operational Handover

### For the next developer/data scientist

11. 1. Clone or open the project repository.
12. 2. Create/activate the Python environment.
13. 3. Install the dependencies from `requirements.txt`.
14. 4. Confirm that the model package exists under `models/`.
15. 5. Confirm that the preprocessing features match the saved model feature list.
16. 6. Run the Streamlit application.
17. 7. Test several representative legitimate and high-risk scenarios.
18. 8. Verify that probabilities are returned without preprocessing errors.
19. 9. Confirm that unseen categorical values are handled by the fitted encoder.
20. 10. Record any production changes in version control.

### Model handover

The primary model is:

The deployment package contains:

The operating threshold is:

## 21. Important Deployment Considerations

### 21.1 Feature consistency

The most important deployment requirement is that the application generates features in exactly the same way as the training pipeline.

Any change to:

- feature names,
- category values,
- feature calculations,
- feature ordering,
- preprocessing,

can affect model predictions.

### 21.2 Negative transaction amounts

The underlying project identified negative transaction amounts and unusual negative 30-day averages during data quality analysis.

The current Streamlit interface sets the transaction amount input to a minimum of zero. Therefore, the current UI does not support entering negative transaction amounts even though negative values exist in the underlying dataset.

If refunds, reversals or adjustments must be scored in production, this should be reviewed before deployment.

### 21.3 High-velocity threshold

The Streamlit application creates ``high_velocity_flag`` when ``transaction_velocity_1h >= 5``.

This threshold should be treated as a project feature-engineering rule, not as a universally valid fraud threshold.

### 21.4 Automatic blocking

The prototype should not automatically block transactions.

The intended action is:

## 22. Analyst Feedback Loop

A production-oriented extension is to record analyst decisions:

This allows the system to evolve as new fraud patterns appear.

The analyst outcome data should be stored separately from the original training dataset and incorporated into retraining only after appropriate quality checks.

## 23. Monitoring Recommendations

Before production deployment, monitor:

### Data drift

Check whether the distribution of important features changes over time.

Examples:

- Transaction amount.
- Amount-to-average ratio.
- Transaction velocity.
- New-device rate.
- Cross-border rate.

### Prediction drift

Monitor changes in the distribution of fraud probabilities.

### Model performance

When confirmed analyst labels become available, monitor:

- Precision.
- Recall.
- F1-score.
- ROC-AUC.
- False-positive rate.
- Fraud detection rate.

### Operational metrics

Also monitor:

- Number of transactions reviewed.
- Number of high-risk alerts.
- Analyst workload.
- Confirmed-fraud percentage.
- False-positive percentage.
- Average investigation time.

## 24. Limitations

The current solution has several limitations:

21. 1. It is a prototype rather than a production fraud platform.
22. 2. The historical dataset may not represent future transaction behaviour.
23. 3. SMOTE creates synthetic training examples and does not create new real-world fraud observations.
24. 4. The model has not been tested in a live payment-processing environment.

25. 5. Production retraining infrastructure has not been implemented.
  26. 6. Real-time feature stores and live account-level velocity calculations are outside the current scope.
  27. 7. The current Streamlit interface is designed for demonstration and analyst assessment rather than high-volume production scoring.
  28. 8. Model probability calibration has not been formally validated.
  29. 9. The 0.50 threshold has not been optimised using production fraud-loss and analyst-capacity data.
  30. 10. Feature importance indicates predictive contribution, not causation.
11. Potential data-generation issues, including unusual negative 30-day averages, require further investigation.
12. The current interface does not support negative transaction amounts even though such values exist in the underlying data.

## 25. Recommended Future Improvements

### Short term

- Add batch CSV scoring to Streamlit.
- Add downloadable analyst review queues.
- Add Low/Medium/High/Critical visual summaries.
- Add model confidence/probability distributions.
- Add analyst feedback capture.
- Add automated model and data validation checks.
- Freeze and document the final dependency versions.

### Medium term

- Calibrate Random Forest probabilities.
- Compare PR-AUC in addition to ROC-AUC.
- Evaluate alternative imbalance methods.
- Perform cross-validation.
- Tune hyperparameters.
- Add explainability such as SHAP.
- Implement drift monitoring.
- Validate thresholds against actual fraud-loss costs and analyst capacity.

### Long term

- Integrate with a real-time transaction stream.
- Build a feature store for account behaviour and velocity.
- Implement controlled model retraining.
- Introduce model versioning.
- Add audit logging.
- Implement role-based analyst access.

- Integrate confirmed analyst outcomes into the model monitoring lifecycle.

## 26. Final Business Value

The prototype demonstrates how Finlora can move from a simple binary fraud-rule mindset toward a risk-based transaction-monitoring approach.

Instead of asking only:

> "Is this transaction fraudulent?"

the system asks:

> "How likely is this transaction to be fraudulent, and should an analyst review it?"

This allows fraud teams to prioritise investigation resources around transactions with higher estimated risk.

# Handover Summary

## Project status

Status: Prototype completed / ready for demonstration and controlled testing.

### Completed

- Data cleaning.
- Data-quality investigation.
- Exploratory analysis.
- Feature engineering.
- Velocity validation.
- Stratified train/test split.
- One-hot encoding.
- SMOTE training-data balancing.
- Logistic Regression baseline.
- Random Forest prototype.
- Model evaluation.
- Confusion matrices.
- Feature-importance analysis.
- Fraud probability scoring.
- Random Forest deployment package.
- Streamlit transaction-risk interface.

### Primary model

Random Forest

### Baseline

Logistic Regression

## Current operating threshold

0.50 fraud probability

## Intended action

\*\*Prioritise high-risk transactions for analyst review.

## Not implemented

- Automatic transaction blocking.
- Live payment integration.
- Production feature store.
- Production model monitoring.
- Automated retraining.
- Formal probability calibration.
- Production-grade authentication/authorisation.

## # Final Handover Checklist

- ☒ Cleaned data prepared.
- ☒ EDA completed.
- ☒ Features engineered.
- ☒ Velocity independently validated.
- ☒ 75/25 stratified split completed.
- ☒ SMOTE applied to training data only.
- ☒ Logistic Regression trained.
- ☒ Random Forest trained.
- ☒ Models evaluated.
- ☒ Random Forest selected as primary prototype.
- ☒ Deployment package created.
- ☒ Streamlit application created.
- ☐ Final requirements.txt frozen.
- ☐ Batch scoring added.
- ☐ Analyst feedback workflow fully integrated.
- ☐ Production monitoring implemented.
- ☐ Probability calibration completed.
- ☐ Production threshold validated using business cost and review capacity.
- ☐ Live transaction integration completed.

---

## Final handover statement

Finlora Finance Flow's fraud-risk scoring prototype is complete and demonstrates an end-to-end machine-learning workflow from data preparation through model development and interactive risk assessment. Random Forest is the selected primary prototype because it provides the strongest overall balance of accuracy, precision and F1-score, while Logistic Regression provides an interpretable

benchmark. The current system should be treated as a decision-support prototype for analyst review, not as an autonomous transaction-blocking system.